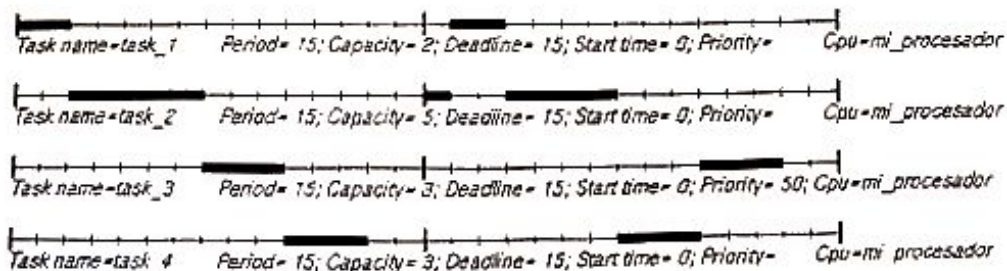


**Sistemas de Tiempo Real - Parcial, 2^{da} fecha.- 12/12/2018***Para aprobar el examen es necesario realizar todos los ejercicios.*

1) Realice un programa que adquiera un valor de alguna de las entradas analógicas de la placa ADQ12 una vez por segundo y lo imprima por pantalla, de manera similar a como lo hizo en la práctica 1. Pero cuando el valor del muestreo supere un umbral debe pasar a adquirir 4 valores por segundo. Para medir los intervalos de tiempo se debe programar la interrupción del timer. Comente la función que realizan las instrucciones de su programa.

2) Realice un programa, similar a los de la práctica 2, utilizando un microcontrolador con tres salidas digitales (cada una conectada a un led) y una entrada conectada a un switch. Controle los leds como si se tratara de un semáforo y al switch como un botón para peatones. Para los tiempos de espera debe implementar un procedimiento del tipo *espera(int n)* que recibe como parámetro la cantidad de milisegundos a esperar (el botón de peatón tiene distinto tiempo de espera que el del semáforo). Agregue comentarios a las instrucciones de su programa.

3) Complete las prioridades de las tareas 1, 2 y 4. Además, complete la política de cada una de las tareas. Realice una explicación del porqué de su respuesta.



4) Detalle el encabezado de la función de creación de tareas en FreeRTOS, enumerando y explicando cada uno de sus parámetros.

5) Realice un programa utilizando Pthreads, en el cual se deben crear 2 hilos con la misma prioridad. Cada hilo debe imprimir solo un número de identificación (1, 2). La ejecución de los hilos debe mostrar constantemente en pantalla la siguiente secuencia: 2 - 2 - 1

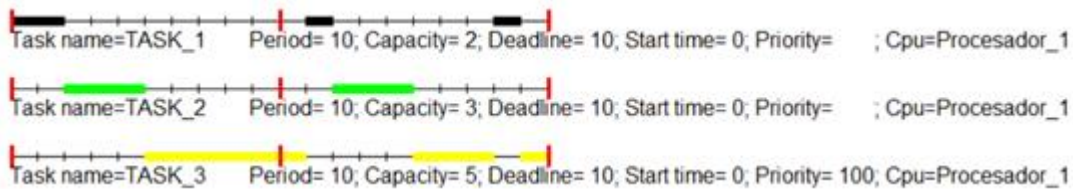
Sistemas de Tiempo Real - Parcial, 1^{ra} fecha.- 21/10/2020

Para aprobar el examen es necesario realizar todos los ejercicios

Práctica

1) Realice un programa, similar a los de la práctica 1, utilizando un microcontrolador con tres salidas digitales (cada una conectada a un led) y una entrada conectada a un switch. Controle los leds como si se tratara de un semáforo peatonal y al switch como un botón para que peatones con dificultades (ciegos, problemas motrices, etc) puedan cruzar. Para los tiempos de espera debe implementar un procedimiento del tipo *espera(int n)* que recibe como parámetro la cantidad de milisegundos a esperar (el botón de peatón tiene distinto tiempo de espera que el de las luces del semáforo funcionando normalmente). Agregue comentarios a las instrucciones de su programa.

2) Complete las prioridades de las tareas 1y 2. Además, complete la política de cada una de las tareas y el quantum. Realice una explicación del porqué de su respuesta.



Sistemas de Tiempo Real – Parcial 2, 1^{ra} fecha. - 09/12/2020

Para aprobar el examen es necesario realizar todos los ejercicios

Práctica

1) Desarrolle un ejercicio sobre FreeRTOS, en el cual tres tareas trabajan sobre la misma variable global para realizar diferentes acciones.

Una de las 3 tareas tiene mayor prioridad, se ejecuta cada 100ms y su función es incrementar en uno el valor de la variable global.

Las otras dos tareas tienen una prioridad menor a la tarea uno, pero igual entre sí, y su función es imprimir el valor de la variable global por consola cada 50ms; luego de imprimir deben incrementar en dos el valor de la variable global.

Aclaración: No hay que considerar la posibilidad de que la variable desborde por tipo de dato.

2) a- Enuncie y describa la función y los parámetros para la creación de hilos disponible en Posix Pthread.

b- Enuncie y describa las funciones para la creación y el manejo de semáforos binarios en Posix Pthread.

Sistemas de Tiempo Real - Parcial, 1ª fecha - 23/11/2022

Para aprobar el examen es necesario realizar todos los ejercicios

Práctica

1) Realice un programa, similar a los de la práctica 1, utilizando un microcontrolador con un puerto de ocho salidas digitales (cada una conectada a un led) y otro puerto con una entrada conectada a un switch.

Al pulsar el switch se debe iluminar la posición menos significativa y realizar un desplazamiento de la iluminación de los leds de derecha a izquierda cada 250 ms. El tiempo se calcula e implementa utilizando el timer0 y su interrupción respectiva. Una vez completa la secuencia (se iluminó el led más significativo) se reinicia desde el led menos significativo. Si se vuelve a pulsar el switch, se reanuda la secuencia.

2) Realice el diagrama de planificación generado por el algoritmo POSIX 1003.1b/Highest Priority First para los dos primeros períodos teniendo un quantum de 2 (dos).

	Policy	Priority	Capacity	Deadline	Period
TAREA 1	RR	100	2	10	10
TAREA 2	RR	100	3	10	10
TAREA 3	RR	100	5	10	10

3) Desarrolle un ejercicio sobre FreeRTOS, en el cual tres tareas trabajan sobre la misma variable global para realizar diferentes acciones.

Una de las 3 tareas tiene mayor prioridad, se ejecuta cada 100ms y su función es incrementar en uno el valor de la variable global.

Las otras dos tareas tienen una prioridad menor a la tarea uno, pero igual entre sí. Su función es imprimir una identificación de la tarea y el valor de la variable global por consola cada 50ms; luego de imprimir deben incrementar en dos el valor de la variable global.

Aclaración: No hay que considerar la posibilidad de que la variable desborde por tipo de dato.

4) Enuncie y describa las funciones para la creación y el manejo de mutex en Posix Pthread.

1. Frec. del reloj = 1MHz

Periodo del reloj = 1 μ s $\Rightarrow$ Ciclo de instr. = 4 periodos

Periodo de int. = 4 μ s

$$\frac{250ms}{4\mu s} = 62,5 \cdot 10^3$$

$$\log_2\left(\frac{62500}{256}\right) \approx 8$$

Valor de prescaler

int LEDact = 0;

int main() {

TRISA = 0b00000000; // Configuro Todos los pines RA como salida

TRISB = 0b00000001; // RBO entrada

TOIE = 1; // Habilito la int. del clock

PSA = 0; // Habilito el uso del prescaler

T0CS = 0; // Uso el reloj interno

PS0 = 1; PS1 = 1; PS2 = 1; // PS = 111₂ + 1 = 7 + 1 = 8 $\Rightarrow$ 1:256

TMR0 = 0;

GIE = 0; // Todavía no habilito interrupciones

while (True) {

if (RBO) { // Si se activa el switch

RA = 0b00000001;

// Reinicio los LEDs

LEDact = 0;

if (!GIE) GIE = 1;

// si no estaba activo, habilito

// Interrupciones

delay_ms(100);

// Para no sobrecargar

// el microcontrolador

} return 0;

// Manejador de interrupción del Timer 0

void interrupt Timer (void) {

TOIE = 0;

// Deshabilito la interrupción

LEDact = (LEDact + 1) % 8; // Calculo el sig. LED

RA = 1 << LEDact;

// Asigno el LED actual a la salida

TMRO = 0;

// Reinicio el contador

TOIF = 0;

// Reinicio flag de overflow

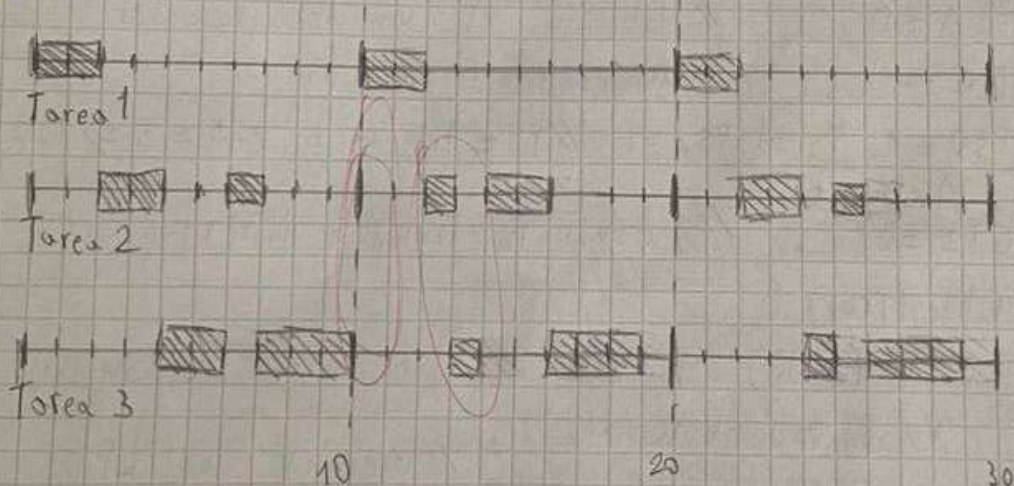
TOIE = 1;

// Vuelvo a habilitar la interrupción

}

- HAY CÓDIGO QUE PODRÍA ESTAR EN EL MAIN
- MAL CALCULO TMR0

2. Q=2



3. #Include <Arduino_FreeRTOS.h>

int global=0;

xSemaphoreHandler mutex;

void Task1 ((*void) PVParameters) {

while (true) {

if (xSemaphoreTake(mutex, 10) == PVTrue) {

global++;

xSemaphoreGive(mutex);

}

vTaskDelay(100/port-TICK-PERIOD-MS);

}

}

void Task2 ((*void) PVParameters) {

int id = (int) PVParameters;

while (true) {

if (xSemaphoreTake(mutex, 10) == PVTrue) {

Serial.println(id);

Serial.println(global);

global+=2;

xSemaphoreGive(mutex);

}

vTaskDelay(50/port-TICK-PERIOD-MS);

}

}


```

void Setup() {
    Serial.Begin(9600); mutex = xMutexCreate();
    xTaskCreate(Task1, "Tarea 1", 128, NULL, 2, NULL); // Prioridad más alta
    xTaskCreate(Task2, "Tarea 2", 128, 2, 1, NULL);
    xTaskCreate(Task2, "Tarea 3", 128, 3, 1, NULL);
}
    → FALTA INICIALIZAR SCHEDULER
void Loop() {}

```

- 4 -
- `pthread_mutex_init(&mutex);` → **ATTR**
 - ↳ Permite crear un semáforo mutex de tipo `pthread_mutex_t`
 - `pthread_mutex_lock(&mutex);`
 - ↳ Permite apropiarse del mutex [Ecuivalente a un `P(mutex)`], si el semáforo está bloqueado, el hilo queda suspendido hasta que sea desbloqueado.
 - `pthread_mutex_unlock(&mutex);`
 - ↳ Permite desbloquear un semáforo previamente bloqueado por el hilo que invocó esta función [Ecuivalente a `V(mutex)`]

Sistemas de Tiempo Real - Parcial, 1^{ra} fecha - 13/10/2021

Para aprobar el examen es necesario realizar todos los ejercicios

Práctica

1) Realice un programa, similar a los de la práctica 1, utilizando un microcontrolador con un puerto de ocho salidas digitales (cada una conectada a un led) y otro puerto con una entrada conectada a un switch.

Al pulsar el switch se debe iluminar la posición menos significativa y realizar un desplazamiento de la iluminación de los leds de derecha a izquierda cada 250 ms. El tiempo se calcula e implementa utilizando el timer0 y su interrupción respectiva. Una vez completa la secuencia (se iluminó el led más significativo) se reinicia desde el led menos significativo. Si se vuelve a pulsar el switch, se reanuda la secuencia.

2) Realice el diagrama de planificación generado por el algoritmo POSIX 1003.1b/Highest Priority First para los dos primeros períodos teniendo un quantum de 2 (dos).

	Policy	Priority	Capacity	Deadline	Period
TAREA 1	RR	100	2	10	10
TAREA 2	RR	100	3	10	10
TAREA 3	RR	100	5	10	10



Sistemas de Tiempo Real – Parcial 2, 1^{ra} fecha. - 15/12/2021

Para aprobar el examen es necesario realizar todos los ejercicios

Práctica

1) Desarrolle un ejercicio sobre FreeRTOS, en el cual tres tareas trabajan sobre la misma variable global para realizar diferentes acciones.

Una de las 3 tareas tiene mayor prioridad, se ejecuta cada 100ms y su función es incrementar en uno el valor de la variable global.

Las otras dos tareas tienen una prioridad menor a la tarea uno, pero igual entre sí. Su función es imprimir una identificación de la tarea y el valor de la variable global por consola cada 50ms; luego de imprimir deben incrementar en dos el valor de la variable global.

Aclaración: No hay que considerar la posibilidad de que la variable desborde por tipo de dato.

2) Enuncie y describa las funciones para la creación y el manejo de mutex en Posix Pthread..

3) Desarrolle un ejercicio utilizando Posix Pthread que genere 3 hilos con la misma prioridad. Los hilos deben ejecutarse mil veces cada uno y en cada iteración deben imprimir en consola su identificación el número de iteración en la que están.